Protection des données sous SQL Server:

Les algorithmes et les différentes méthodes de chiffrement et déchiffrement des données

Pathé Djibril DIOUF

PLAN:

I. INTRODUCTION
II. CONTEXTE ET MOTIVATION
III. PROBLÉMATIQUE
IV. OBJECTIFS
V. OUTILS UTILISÉS
VI. MÉTHODES ET SOLUTIONS PROPOSÉES
VII. AVANTAGES ET INCONVÉNIENTS
VIII. PRÉSENTATION ET DÉPLOIEMENT DES SOLUTIONS
IX. CONCLUSION ET PERSPECTIVES
X. BIBLIOGRAPHIE ET WEBOGRAPHIE

I. Introduction

Les données sont l'actif le plus précieux de toute entreprise quel que soit le secteur d'activité. Aujourd'hui, les entreprises doivent gérer d'énormes volumes de données et cela tout en garantissant la confidentialité, l'intégrité et la disponibilité des données.

L'utilisation d'une base de données semble être alors un excellent choix, dans la mesure où, une base de données est un ensemble d'informations organisé de sorte à être facilement accessible, géré et mis à jour.

Il existe à cet effet plusieurs types de base de données(hiérarchiques, relationnelles, …). SQL Server fait partie des bases de données relationnelles les plus utilisées actuellement.

II. Contexte et motivation

La protection des données concerne les pratiques actives de sécurité. Elle nécessite des outils et des procédures permettant de protéger les données contre les accès non autorisés, les modifications, la divulgation accidentelle, les perturbations et la destruction.
Cela implique d'utiliser des stratégies physiques et logiques pour protéger les informations contre les violations de données, les cyberattaques ou les pertes de données accidentelles ou intentionnelles.

SQL Server utilise des clés de chiffrement pour sécuriser les données, les informations d'identification et les informations de connexion stockées dans une base de données serveur. SQL Server a deux types de clés : symétrique et asymétrique.

III. Problématique

La protection des données est une préoccupation de plus en plus importante au vu des cyberattaques tels que les ransomwares qui deviennent de plus en plus fréquentes. Ainsi, elle est un élément important de la stratégie de sécurité globale.

Il devient alors important de s'interroger sur :

- l'impact du chiffrement SQL Server sur la rapidité d'accès à l'information ?
- la volumétrie de stockage des données chiffrées ?

IV. Objectifs

La protection par le chiffrement est un très précieux pour garantir la sécurité des données. En effet le chiffrement a pour objectif de rendre les données inintelligibles à l'aide d'une clé ou d'un mot de passe.

Avec SQL Server, le chiffrement inclut une combinaison de certificats, clés publiques, privées et symétriques utilisées pour protéger les données sensibles. En effet, la clé symétrique est utilisée par SQL Server pour chiffrer les données sensibles stockées dans SQL Server. Les clés publique et privée sont créées par le système d'exploitation et servent à protéger la clé symétrique.

Ainsi, il devient nécessaire d'appliquer plusieurs fonctionnalités de protection de données visant des étendues de sécurité différentes.

V. Outils utilisés

Pour les différentes solutions que nous allons aborder dans la suite, nous travaillerons avec la version **Developer de SQL Server 2016 .**
Nous utiliserons SQL Server Management Studio (SSMS) v18.12.1

VI. Méthodes et solutions proposées

SQL Server a deux applications principales pour les clés : **une clé principale de service** (Service Master Key) (**SMK**) générée et pour une instance SQL Server, ainsi qu'une **clé principale de base de données** (Database Master Key) (**DMK**) utilisée pour une base de données.

SMK est généré automatiquement la première fois que l'instance de SQL Server est démarrée et est utilisée pour chiffrer un mot de passe de serveur lié, des informations d'identification et la clé principale de base de données dans chaque base de données.

DMK est une clé symétrique qui permet de protéger les clés privées des certificats et des clés asymétriques présentes dans la base de données.

L’image suivante décrit l’hiérarchie des clés de chiffrement et les certificats fournie par SQL Server.

L’API de protection des données Windows (DPAPI) se trouve à la racine de l’arborescence de chiffrement, sécurise la hiérarchie de clés au niveau de l’ordinateur et est utilisée pour protéger la clé principale de service (SMK) pour l’instance du serveur de base de données.

En effet, le **SMK** protège le **DMK**, qui est stockée au niveau de la base de données utilisateur et protège les **certificats** et les **clés asymétriques**. Ces derniers, à leur tour, protègent les **clés symétriques**, qui protègent les **données**.

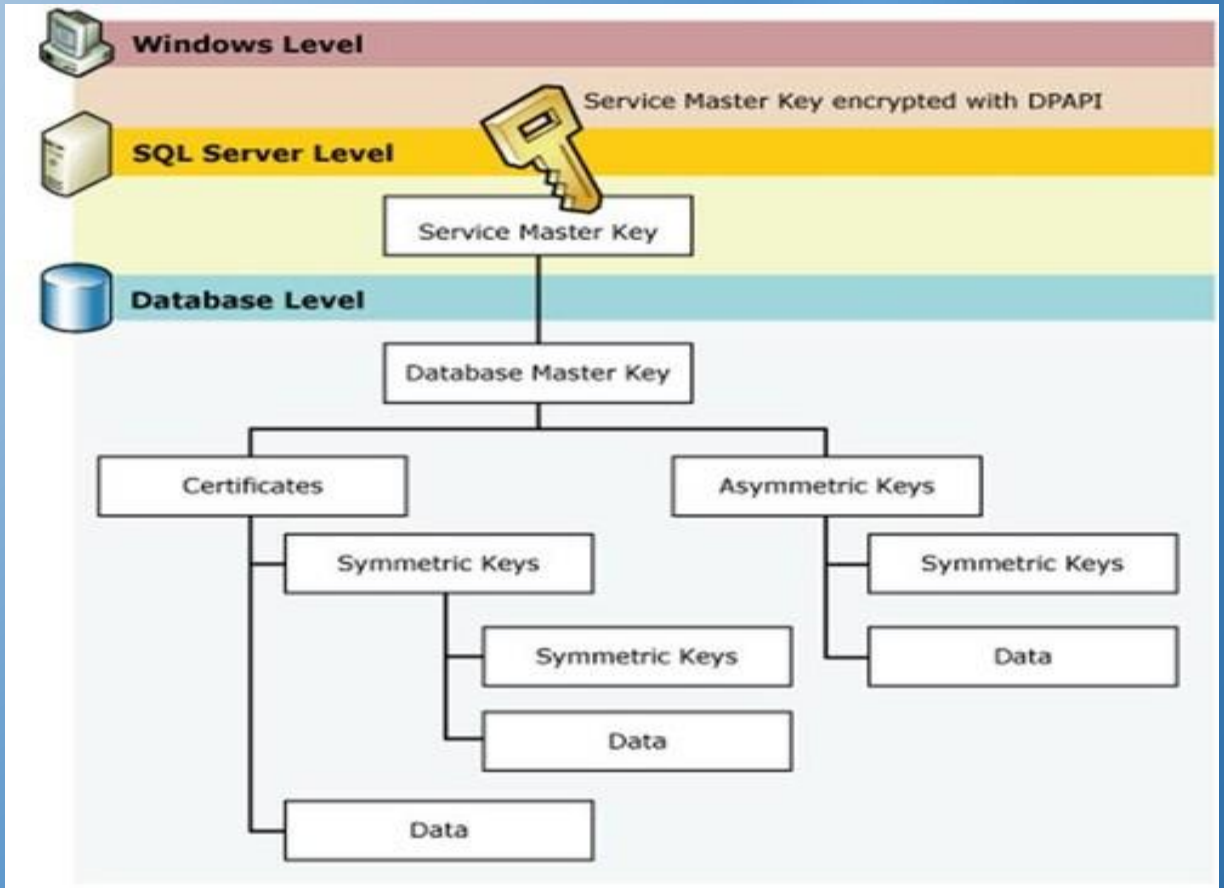

Figure 1 : Hiérarchie de chiffrement de SQL Server [W1]

1) Les algorithmes de chiffrement

SQL Server permet aux administrateurs et aux développeurs de choisir entre plusieurs algorithmes, notamment:

- **DES**
- **Triple_DES(3DES)**
- **TRIPLE_DES_3KEY**
- **RC2,**
- **RC4,**
- **RC4_128 bits**
- **DESX**
- **AES_128 bits**
- **AES_192 bits**
- **AES_256 bits**

NB:

A partir de SQL Server 2016, tous les algorithmes autres que **AES_128, AES_192 et AES_256** sont déconseillés.

En effet, SQL Server utilise l'algorithme de chiffrement AES pour protéger la clé principale de service (SMK) et la clé principale de base de données (DMK). AES est un algorithme de chiffrement plus récent que 3DES, qui était utilisé dans les versions antérieures. Après la mise à niveau d'une instance du moteur de base de données vers SQL Server SMK et DMK doivent être régénérés afin de mettre à niveau les clés principales vers AES.

b) Les méthodes de chiffrement et de déchiffrement

Plusieurs solutions s'offrent à nous.
En effet, sous SQL Server il existe plusieurs méthodes de chiffrement :

1) **Chiffrement du backup**
2) **Chiffrement transparent de données (TDE)**
3) **Chiffrement au niveau d'une colonne avec ALWAYS ENCRYPTED**
4) **Chiffrement au niveau des lignes de table (SNL)**

A noter qu'il existe également une autre méthode qui, à proprement parlé, n'est pas du chiffrement mais plutôt du masquage.

5) **Le Masquage dynamique de données (DDM)**

VII. Avantages et Inconvénients

1) Chiffrement du backup

- Avantages :

- En cas de perte accidentelle de données, on peut les restaurer dans l'état antérieur où elles étaient.
- Le chiffrement sert également pour les bases de données qui sont chiffrées à l'aide du chiffrement TDE.
- Vous pouvez intégrer des clés de chiffrement avec les fournisseurs EKM (Gestion de clés extensible).

- Inconvénients:

- Si vous utilisez une clé asymétrique pour chiffrer les données de sauvegarde, seules les clés asymétriques résidant dans le fournisseur EKM sont prises en charge.

- Les versions précédentes de SQL Server ne peuvent pas lire les sauvegardes chiffrées.

2) Chiffrement transparent des données (TDE) :

- Avantages:

- Sans le TDE, un attaquant peut prendre le support physique (lecteurs ou bandes de sauvegarde) et restaurer ou joindre la base de données afin de lire le contenu.

- Inconvénients:

- Le TDE n’assure pas de chiffrement sur les canaux de communication.
- Si une base de données utilise TDE, alors la base de données TEMPDB sera également chiffrée. Ce chiffrement peut avoir des conséquences sur le niveau de performance des bases de données non chiffrées sur la même instance SQL Server.
- Il nécessite de chiffrer toute la base de données.
- Avec le TDE, toutes les données sont chiffrées de la même manière.

3) Chiffrement au niveau des colonnes avec ALWAYS ENCRYPTED

- Avantages :

- ❑ Les données sont chiffrées au repos, en mouvement et en mémoire.
- ❑ On peut choisir de chiffrer une seule colonne.
- ❑ TEMPDB n’est pas chiffré.
- ❑ Deux choix de chiffrement : Déterministe (pour les indexes) et Aléatoire.

- <u>Inconvénients :</u>

- Le chiffrement déterministe ne prend en charge que les opérations suivantes impliquant les comparaisons d'égalité.
  - = (égal) dans les recherche de points
  - IN
  - SELECT – GROUP BY
  - DISTINCT

- Le chiffrement aléatoire empêche la jointure sur des colonnes chiffrées.

4) Chiffrement au niveau des lignes de table (Sécurité au Niveau des Lignes)

- Avantages :

- ❑ Implémenter des restrictions sur l'accès aux lignes de données.
- ❑ Réduction de la surface d'exposition du système de données.
- ❑ Peut être implémenter avec Always Encrypted ou le DDM pour optimiser la sécurité.

- Inconvénients

- La sécurité au niveau des lignes n’est pas compatible avec des tables qui ont des vues indexées.

- Avec le SNL, on note une petite perte de performance entre l’utilisation de la fonction USER_NAME() et de la fonction SESSION_CONTEXT().

5) Masquage dynamique de données (DDM)

- Avantages :
  - ❑ Empêche les accès non autorisés à des données sensibles
  - ❑ Désignez les utilisateurs ou les rôles privilégiés qui ont accès aux données sensibles.
- Inconvénients:
  - ❑ Impossible de définir une règle de masquage pour les colonnes chiffrées.
  - ❑ Le DDM ne peut pas être appliqué sur une colonnes ayant des dépendances.

VIII. PRÉSENTATION ET DEPLOIEMENT DES SOLUTIONS

1) Chiffrement du backup

La sauvegarde chiffrée permet tout d'abord de sécuriser les données sauvegardées, mais surtout de ne restaurer les sauvegardes chiffrées que sur un serveur ou une instance disposant du certificat ou de la clé de chiffrement asymétrique utilisée.

Pour chiffrer pendant la sauvegarde, vous devez spécifier un algorithme de chiffrement, et un chiffreur pour sécuriser la clé de chiffrement. Les options de chiffrement suivantes sont prises en charge :

- **Algorithme de chiffrement** : les algorithmes de chiffrement pris en charge sont : AES_128, AES_192, AES_256 et Triple_DES.
- **Chiffreur** : un certificat ou une clé asymétrique.

- <u>ÉTAPES</u>

  - ❑ Créer une clé principale de base de données pour la base de données MASTER : La clé principale de base de données est une clé symétrique qui permet de protéger les clés privées des certificats et des clés asymétriques présentes dans la base de données.

  - ❑ Créez un certificat ou une clé asymétrique à utiliser pour le chiffrement de la sauvegarde.

- On va tout d'abord créer une base de données nommée MyTestDB.

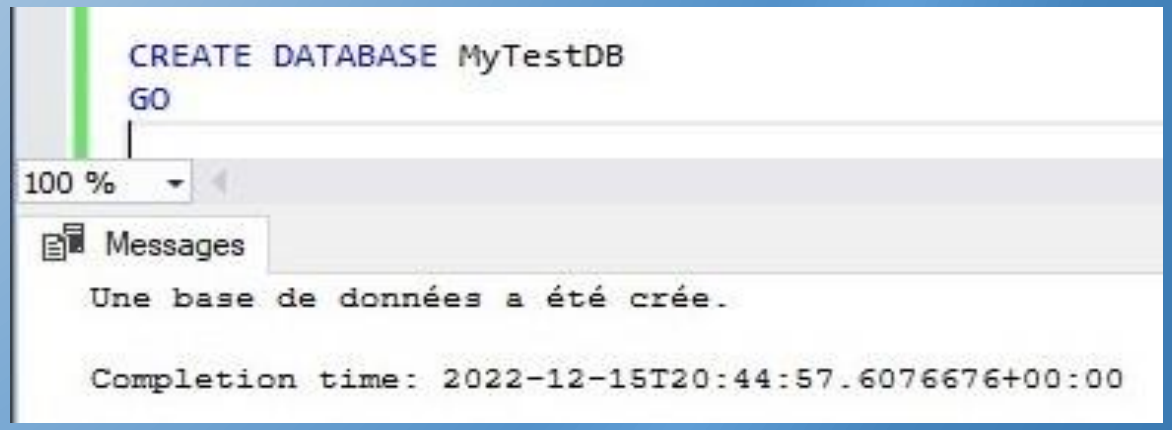

Figure 1.a : Création de la base de données

- La clé stockée dans la base de données est chiffrée par un mot de passe.

```
USE master
GO

CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'P@ssw0rd#123' ;
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-16T00:18:53.2147926+00:00
```

Figure 1.b : Création d’une clé principale dans la base de données master

- Ce certificat est protégé par la clé « master key » créée précédemment.

```
CREATE CERTIFICATE MyTestDBBackupEncryptCert
    WITH SUBJECT = 'MyTestDB Backup Encryption Certificate' ;
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-16T02:21:06.8427070+00:00
```

Figure 1.c : Création d’un certificat de sauvegarde dans la base de données master

- Il est important de faire une sauvegarde du certificat de sauvegarde car sans celui-ci il serait impossible de restaurer la sauvegarde ainsi créée dans cette instance de SQL Server ou dans une autre instance.

```
BACKUP CERTIFICATE MyTestDBBackupEncryptCert
    TO FILE = 'C:\Backup\MyTestDBBackupEncryptCert.cer'
    WITH PRIVATE KEY ( FILE = 'C:\Backup\private_key.key', ENCRYPTION BY PASSWORD = 'P@ssword#123');
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-20T03:19:16.6155558+00:00
```

Figure 1.d : Sauvegarde du certificat

- On procède maintenant au chiffrement de la sauvegarde portant le nom de notre base de données MyTestDB avec l'extension .bak, tout en spécifiant là où l'on veut stocker le fichier, l'algorithme utilisé (AES_256) et le certificat (MyTestDBBackupEncryptCert) ou la clé asymétrique avec lequel on veut chiffrer la sauvegarde.

```sql
BACKUP DATABASE MyTestDB
TO DISK = 'D:\BackupSQL\MyTestDB.bak'
WITH COMPRESSION,
    ENCRYPTION
    (
    ALGORITHM = AES_256,
    SERVER CERTIFICATE = MyTestDBBackupEncryptCert
    ),
    STATS = 10;
GO
```

100 %

Messages

```
11 pour cent traités.
20 pour cent traités.
31 pour cent traités.
40 pour cent traités.
52 pour cent traités.
61 pour cent traités.
70 pour cent traités.
81 pour cent traités.
90 pour cent traités.
352 pages traitées pour la base de données 'MyTestDB', fichier 'MyTestDB' dans le fichier 1.
100 pour cent traités.
2 pages traitées pour la base de données 'MyTestDB', fichier 'MyTestDB_log' dans le fichier 1.
BACKUP DATABASE a traité avec succès 354 pages en 0.027 secondes (102.159 Mo/s).

Completion time: 2022-12-20T03:24:26.4396781+00:00
```

Figure 1.e : Chiffrement de la sauvegarde

- Pour s’assurer que la sauvegarde créée est belle et bien chiffrée. On va supprimer le certificat qui protège la base de données, ensuite la clé principale de base de données, la base de données MyTestDB pour enfin essayer de restaurer la base de données via la sauvegarde chiffrée mais sans le certificat.

```
DROP CERTIFICATE MyTestDBBackupEncryptCert

DROP MASTER KEY

DROP DATABASE MyTestDB
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-20T04:05:57.8678007+00:00
```

Figure 1.f : Suppression du certificat

- On voit bien qu'il est impossible de restaurer la sauvegarde d'une base de données chiffrée sans le certificat ou la clé asymétrique avec laquelle elle a été chiffrée.

```
RESTORE DATABASE MyTestDB FROM DISK = 'D:\BackupSQL\MyTestDB.bak'
GO
```

100 %

Messages

```
Msg 33111, Level 16, State 3, Line 38
Impossible de trouver le serveur certificat avec l'empreinte numérique '0xD352404951CF0F222C3F5D006296C14448E727E9'.
Msg 3013, Level 16, State 1, Line 38
RESTORE DATABASE s'est terminé anormalement.

Completion time: 2022-12-20T04:12:54.5298976+00:00
```

Figure 1.g : Restauration de la base de données

2) Chiffrement TDE

Le Transparent Data Encryption (TDE) protège les données au niveau du fichier en fournissant le chiffrement au repos aux fichiers de base de données,

Le TDE permet de s'assurer que les fichiers de base de données, les fichiers de sauvegarde et les fichiers tempdb ne peuvent pas être joints ni lus sans les certificats corrects pour déchiffrer les fichiers de base de données.

Le chiffrement d'un fichier de base de données s'effectue au niveau de la page. Les pages au sein d'une base de données chiffrée sont chiffrées avant d'être écrites sur disque et déchiffrées au moment d'être lues en mémoire. TDE n'augmente pas la taille de la base de données chiffrée.

- <u>ÉTAPES</u>

Pour utiliser le chiffrement TDE, procédez comme suit:

- ❑ Créez une clé principale dans « master »
- ❑ Créez ou procurez vous un certificat protégé par la clé principale
- ❑ Créez une clé de chiffrement(DEK) de base de données et protégez-la à l'aide du certificat.
- ❑ Configurez la base de données pour qu'elle utilise le chiffrement.

- On crée en premier lieu la base de données TDEDatabase avec laquelle on va travailler dans cette partie.

```
CREATE DATABASE TDEDatabase
GO
```

100 %

Messages

```
Une base de données a été crée.

Completion time: 2022-12-16T23:53:19.6089085+00:00
```

Figure 2.a : Création d'une base de donnée

- On crée une clé principale de base de données protégée par un mot de passe dans la base de données « master ».

```
USE master
GO

CREATE MASTER KEY ENCRYPTION BY PASSWORD = '#motdep@$$eFort#';
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-17T00:51:57.8346498+00:00
```

Figure 2.b : Création d’une clé principale de base de données

- Ce certificat TDECert est protégé par la clé principal de base de données.

```
CREATE CERTIFICATE TDECert
    WITH SUBJECT = ' TDE Certificate';
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-17T01:06:20.0881565+00:00
```

Figure 2.c : Création d’un certificat

- On fait un backup du certificat avec une clé privée qui est utilisée par le système pour déchiffrer le certificat. Et cette clé privée est elle-même protégée par un mot de passe.

```sql
BACKUP CERTIFICATE TDECert
    TO FILE = 'C:\Backup\TDECert.cer'
    WITH PRIVATE KEY ( FILE = 'C:\Backup\private_key.key', ENCRYPTION BY PASSWORD = '#motdep@$$eFort#');
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-20T05:14:36.4914732+00:00
```

Figure 2.d : Sauvegarde du certificat

- On crée une clé de chiffrement de base de données qui sera protégée par le certificat.

```
USE TDEDatabase
GO

CREATE DATABASE ENCRYPTION KEY
    WITH ALGORITHM = AES_256
    ENCRYPTION BY SERVER CERTIFICATE TDECert;
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-20T05:22:02.2674201+00:00
```

Figure 2.e : Création d’une clé de chiffrement

```sql
USE master
GO

ALTER DATABASE TDEDatabase
    SET ENCRYPTION ON
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-20T05:31:58.8704486+00:00
```

Figure 2.f : Activation du TDE

- Pour s’assurer que la chiffrement TDE a été bien mis en place, on peut consulter la vue de

gestion dynamique (DMV) du système : **SYS.DM_DATABASE_ENCRYPTION_KEYS** pour voir

l’état de chiffrement de la base de données TDEDatabase.

En effet, on a le mapping suivant pour l’état de chiffrement de la base de données :

- **0 = Pas de cryptage.**
- **1 = Non crypté.**
- **2 = Cryptage en cours.**
- **3 = Crypté.**
- **4 = Changement de clé en cours.**
- **5 = Décryptage en cours (désactivation de TDE.**
- **6 = Changement de protection en cours.**

En effet, on a deux enregistrements qui affiche un état de cryptage égale à 3. Et c'est la deuxième (**database_id = 5**) qui correspond à la base de données TDEDatabase.

```
SELECT encryption_state, percent_complete , * from SYS.DM_DATABASE_ENCRYPTION_KEYS;
GO
```

100 %

Results | Messages

	encryption_state	percent_complete	database_id	encryption_state	create_date	regenerate_date	modify_date
1	3	0	2	3	2022-12-20 05:31:58.833	2022-12-20 05:31:58.833	2022-12-20 05:31:58.833
2	3	0	5	3	2022-12-20 05:22:02.253	2022-12-20 05:22:02.253	2022-12-20 05:22:02.253

Figure 2.g : Vérification de l'état de la base de données

- La requête suivante nous montre qu'il s'agit réellement de notre base de données. L'autre base de données est ***tempdb*** avec la database_id = 2.

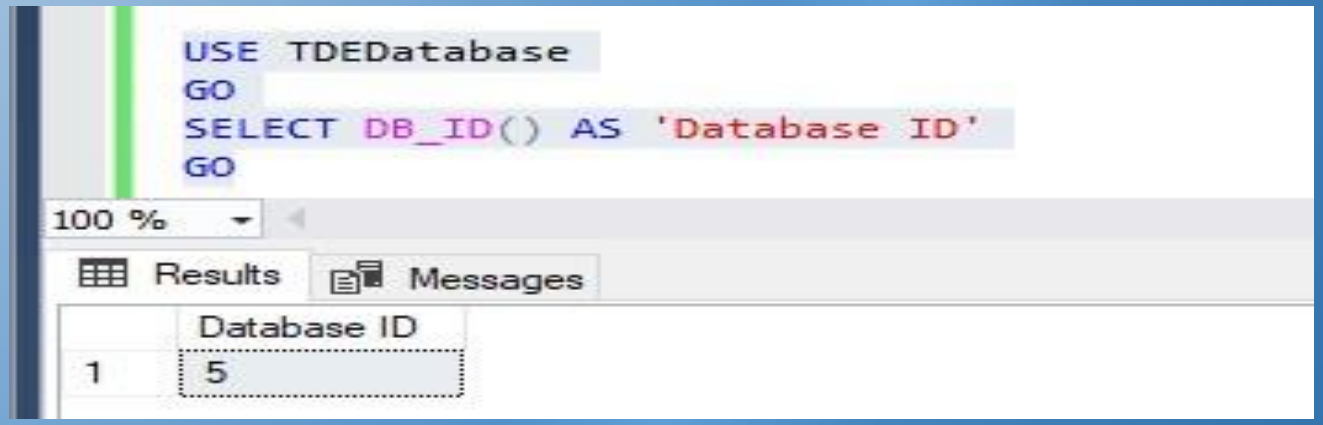

Figure 2.h : Détermination de l'id de la base de données

3) Chiffrement au niveau des colonnes avec ALWAYS ENCRYTED

Pour protéger des données, une base de données n’a pas forcément besoin d’être complètement chiffrée. Il est tout à fait possible de chiffrer uniquement certaines données.

Always Encrypted est une fonctionnalité conçue pour protéger les données sensibles, telles que les numéros de carte de crédit ou les numéros d’identification nationaux (par exemple, les numéros de sécurité sociale), stockées dans la base de données.

Elle permet une séparation entre ceux qui possèdent les données et peuvent les afficher et ceux qui gèrent les données, mais qui ne doivent pas y accéder (administrateurs de base de données locaux, opérateurs de base de données cloud ou autres utilisateurs non autorisés à privilèges élevés).

Always Encrypted utilise deux types de clés :

- Clés de chiffrement de colonne (CEK) :
  Qui sert à chiffrer les données qui se trouvent dans une colonne chiffrée.

- Clés principales de colonne (CMK) :
  Qui est une clé de protection de clé qui chiffre une ou plusieurs clés de chiffrement de colonne.

Les clés principales de colonne dans un magasin de clés approuvé en dehors du système de base de données.

- <u>ÉTAPES</u>

Pour configurer Always Encrypted dans notre base de données, on doit :

- ❑ Provisionnez des clés de chiffrement pour protéger vos données.
- ❑ Configurez le chiffrement pour les colonnes de base de données sélectionnées qui contiennent des données sensibles à protéger.

- On travaille dans cet exemple avec une base de données nommée **AE_Database**, on crée ensuite la table **Employees** dans un schéma **HR.**

```sql
CREATE SCHEMA HR;
GO

CREATE TABLE HR.Employees
(
    EmployeeID int IDENTITY(1,1) NOT NULL
    , SSN char(11) NOT NULL
    , FirstName varchar(50) NOT NULL
    , LastName varchar(50) NOT NULL
    , Salary money NOT NULL
) ;
```

Figure 3.a : Créons un schéma de base de données

```sql
INSERT INTO HR.Employees
(
    SSN
    , FirstName
    , LastName
    , Salary
)
VALUES
(
    '795-73-9838'
    , 'Catherine'
    , 'Abel'
    , $31692
);
```

```sql
INSERT INTO HR.Employees
(
    SSN
    , FirstName
    , LastName
    , Salary
)
VALUES
(
    '990-00-6818'
    , 'Kim'
    , 'Abercrombie'
    , $55415
);
```

Figure 3.b : Insertion de quelques lignes dans la table **Employees**

- Nous allons provisionner une clé principale de colonne et une clé de chiffrement de colonne pour Always Encrypted. Ensuite, nous chiffrerons les colonnes **SSN** et **Salary** dans la table **Employees**

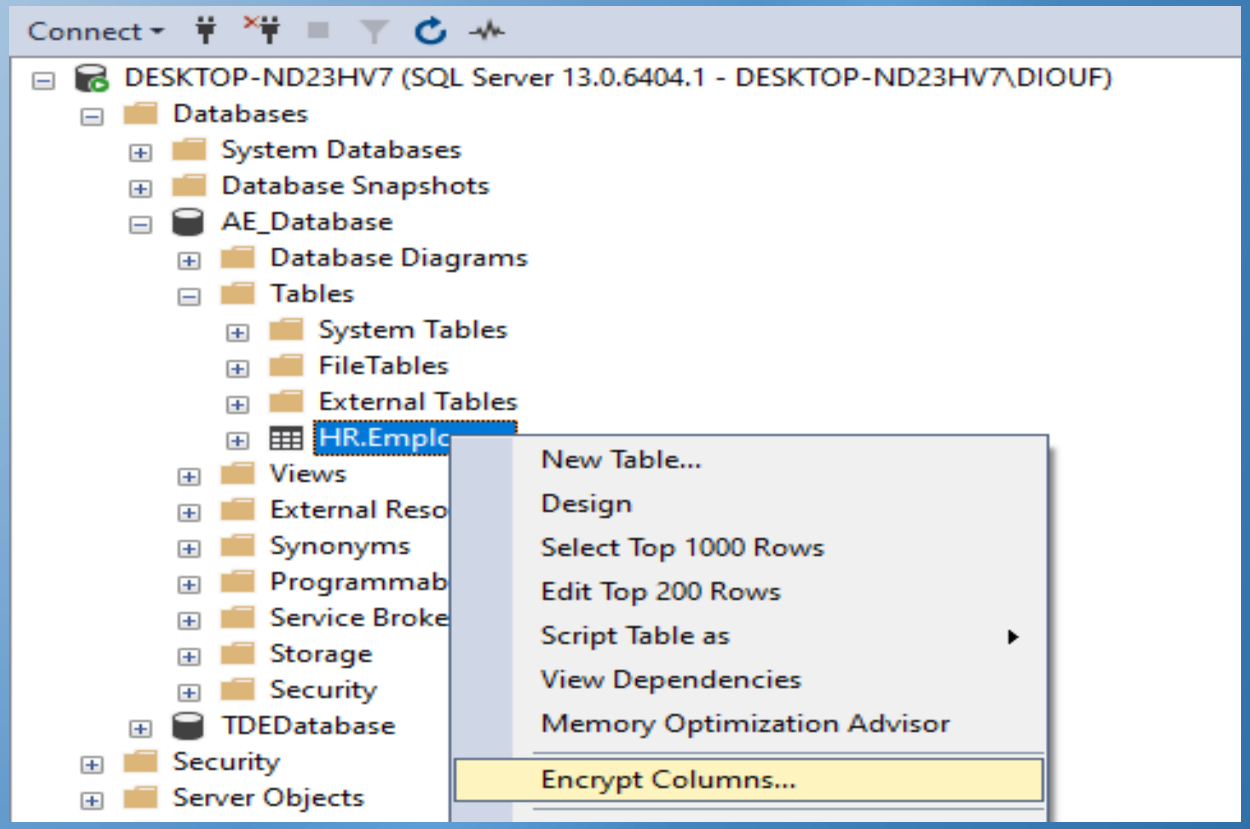

Figure 3.c : Sélection de Always Encrypted

- On choisis le chiffrement déterministe pour la colonne **SSN** et le chiffrement aléatoire pour la colonne **Salary** . Le chiffrement déterministe prend en charge les requêtes, telles que les recherches de recherche de points qui impliquent des comparaisons d'égalité sur des colonnes chiffrées. Le chiffrement aléatoire ne prend pas en charge les calculs sur les colonnes chiffrées.

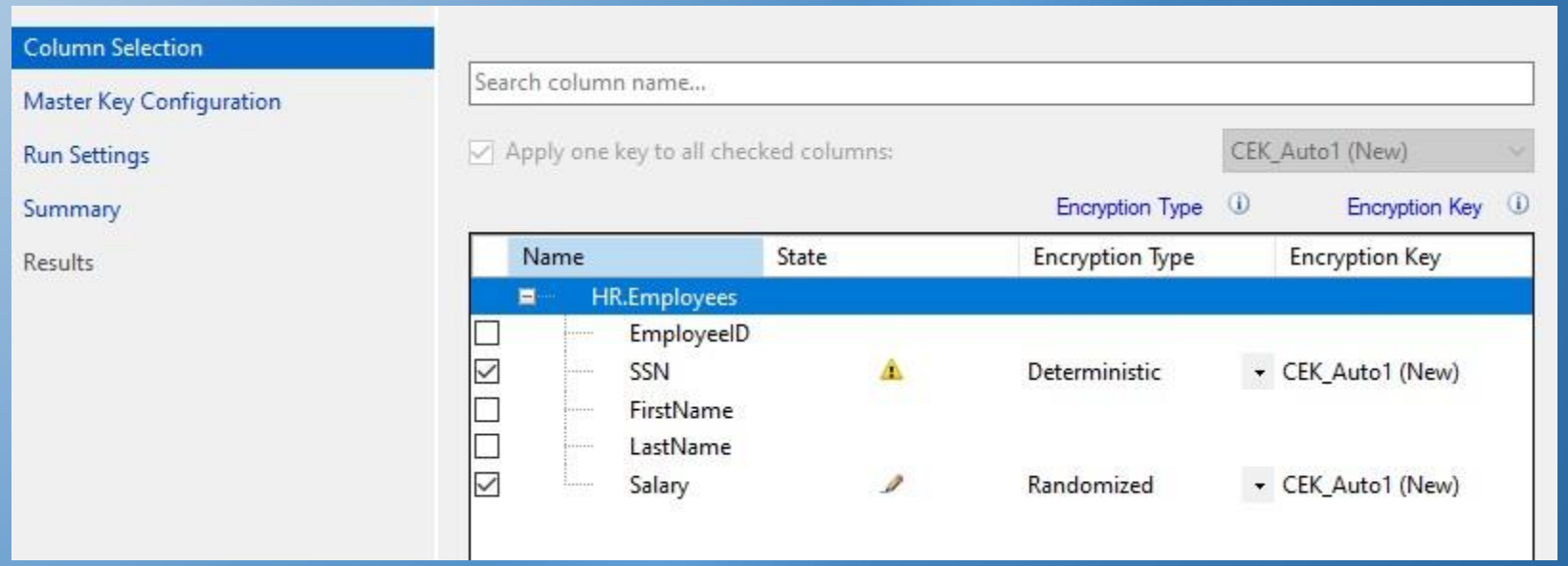

Figure 3.d : Sélection des colonnes à chiffrer

- Dans la page **Configuration de la clé principale** , on configure une nouvelle clé principale de colonne qui sera générée par l'Assistant. Tout d'abord, on sélectionne l'emplacement où l'on souhaite stocker votre clé principale de colonne.

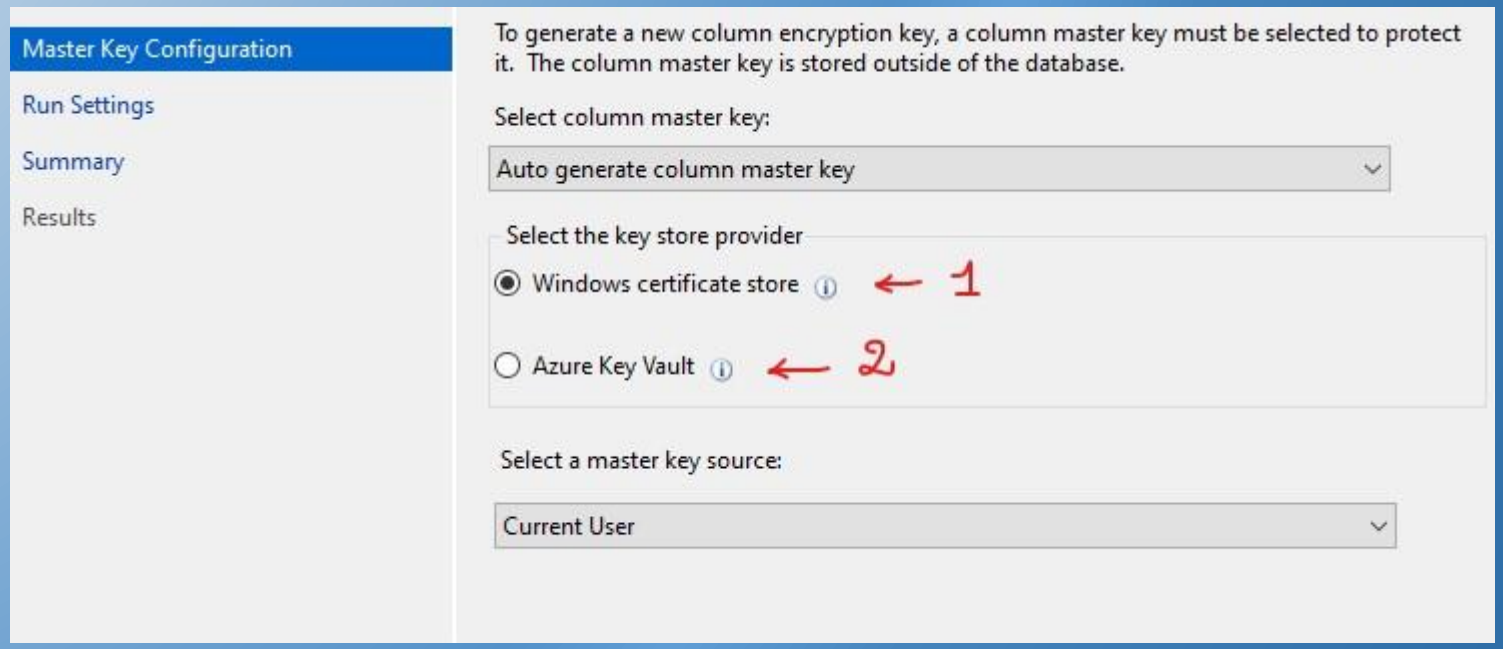

Figure 3.e : Création de la CMK

- Après avoir terminer les dernières étapes, on peut voir que les clés ont été bien crées.

Figure 3.f : Vérification de la création des clés

- Interrogeons la base de données pour voir le chiffrement Always Encrypted

Après s'être connecté à la base de données avec Always Encrypted désactivé, on se rend compte que les colonnes **SSN** et **Salary** sont chiffrées.

Figure 3.g : Connection à la base avec Always Encrypted désactivé

```sql
USE AE_Database
GO

SELECT SSN, Salary FROM HR.Employees;
GO
```

100 %

Results | Messages

	SSN	Salary
1	0x016CE3A3C8B9965BE2598D3C3C1C4DF09E67B045CC2611...	0x01E586BDEC986D0C236683426536C6C29C67D2F08672E23...
2	0x019CA3220C10865AA6D88C757EBA4CF98C63AC221F1E070...	0x01C6BC1A1EBAEABD61A617C79392B1400BC363380D475A...

Figure 3.h : Vérification du chiffrement des colonnes

- Si on réexécute la même requête. Étant donné qu'on est connecté avec Always Encrypted activé pour notre connexion de base de données, le pilote client dans SSMS tente de déchiffrer les données stockées dans les deux colonnes chiffrées.

Figure 3.i : Connection à la base avec Always Encrypted activé

```
USE AE_Database
GO

SELECT SSN, Salary FROM HR.Employees;
GO
```

100 %

Results | Messages

	SSN	Salary
1	795-73-9838	31692,00
2	990-00-6818	55415,00

Figure 3.j : Vérification du déchiffrement des colonnes

4) Chiffrement au niveau des lignes de table (SNL)

Encore appelé la SNL (Sécurité au Niveau des Lignes) permet à la fonctionnalité de tirer profit du contexte d'exécution de l'utilisateur afin de contrôler l'accès aux lignes d'une table de base de données.

La sécurité au niveau des lignes prend en charge deux types de prédicats de sécurité:

- Les prédicats de filtre filtrent en mode silencieux les lignes disponibles pour les opérations de lecture (SELECT, UPDATE et DELETE).

- Les prédicats BLOCK bloquent explicitement les opérations d'écriture (AFTER INSERT, AFTER UPDATE, BEFORE UPDATE et BEFORE DELETE) qui violent le prédicat.

L’accès aux données au niveau des lignes dans une table est restreint par un prédicat de sécurité défini en tant que fonction table incluse. La fonction est ensuite appelée et appliquée par une stratégie de sécurité.

Pour les prédicats de filtre, l’application n’est pas consciente des lignes qui sont filtrées à partir du jeu de résultats. Si toutes les lignes sont filtrées, un jeu de valeurs « *null* » est retourné.

Pour les prédicats BLOCK, toutes les opérations qui violent le prédicat échouent avec une erreur.

Pour illustrer la sécurité au niveau des lignes, on se place dans un scénario où les utilisateurs s’authentifient auprès de la base de données.

Cet exemple crée trois utilisateurs et crée et remplit une table de six lignes. Il crée ensuite une fonction table incluse et une stratégie de sécurité pour la table. L'exemple montre ensuite comment les instructions select sont filtrées pour les différents utilisateurs.

- Créons trois utilisateurs qui illustrent différentes fonctionnalités d'accès

```
CREATE USER Manager WITHOUT LOGIN;
CREATE USER SalesRep1 WITHOUT LOGIN;
CREATE USER SalesRep2 WITHOUT LOGIN;
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-20T23:22:43.4085201+00:00
```

Figure 4.a : Création des utilisateurs

- Créons une table pour stocker les données

```
CREATE SCHEMA Sales
GO
CREATE TABLE Sales.Orders
    (
    OrderID int,
    SalesRep nvarchar(50),
    Product nvarchar(50),
    Quantity smallint
    );
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-20T23:32:50.0336510+00:00
```

Figure 4.b : Création de la table **Sales**

- Remplissons la table avec six lignes de données, en affichant trois commandes pour chaque représentant commercial.

```
INSERT INTO Sales.Orders  VALUES (1, 'SalesRep1', 'Valve', 5);
INSERT INTO Sales.Orders  VALUES (2, 'SalesRep1', 'Wheel', 2);
INSERT INTO Sales.Orders  VALUES (3, 'SalesRep1', 'Valve', 4);
INSERT INTO Sales.Orders  VALUES (4, 'SalesRep2', 'Bracket', 2);
INSERT INTO Sales.Orders  VALUES (5, 'SalesRep2', 'Wheel', 5);
INSERT INTO Sales.Orders  VALUES (6, 'SalesRep2', 'Seat', 5);
GO
```

100 %

Messages

```
(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

Completion time: 2022-12-20T23:38:38.4187988+00:00
```

Figure 4.c : Insertion de données

- Accordons l'accès en lecture sur la table à chaque utilisateur.

```
GRANT SELECT ON Sales.Orders TO Manager;
GRANT SELECT ON Sales.Orders TO SalesRep1;
GRANT SELECT ON Sales.Orders TO SalesRep2;
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-20T23:49:49.9701728+00:00
```

Figure 4.d : Accord de privilège SELECT

- La fonction retourne 1 lorsqu'une ligne de la colonne « SalesRep » est identique à celle de l'utilisateur exécutant la requête (@SalesRep = USER_NAME()) ou si l'utilisateur exécutant la requête est l'utilisateur du Gestionnaire (USER_NAME() = 'Manager').

Cette fonction joue le rôle de filtre pour la stratégie de sécurité.

```
CREATE SCHEMA Security;
GO

CREATE FUNCTION Security.tvf_securitypredicate(@SalesRep AS nvarchar(50))
    RETURNS TABLE
WITH SCHEMABINDING
AS
    RETURN SELECT 1 AS tvf_securitypredicate_result
WHERE @SalesRep = USER_NAME() OR USER_NAME() = 'Manager';
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-21T16:10:55.4831007+00:00
```

Figure 4.e : Création d'un schéma et d'une fonction table inline

- Créons une stratégie de sécurité en ajoutant la fonction comme prédicat de filtre. L'état doit être défini pour ON activer la stratégie.

```
CREATE SECURITY POLICY SalesFilter
ADD FILTER PREDICATE Security.tvf_securitypredicate(SalesRep)
ON Sales.Orders
WITH (STATE = ON);
GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-21T16:30:44.1321632+00:00
```

Figure 4.f : Création de la stratégie de sécurité

- Accordons les autorisations SELECT à la fonction ***tvf_securitypredicate***

```sql
GRANT SELECT ON Security.tvf_securitypredicate TO Manager;
GRANT SELECT ON Security.tvf_securitypredicate TO SalesRep1;
GRANT SELECT ON Security.tvf_securitypredicate TO SalesRep2;
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2022-12-21T16:34:50.7501390+00:00
```

Figure 4.g : Accord de privilège SELECT

- Testons maintenant le prédicat de filtrage, en sélectionnant dans la table **Sales.Orders** comme chaque utilisateur.

  - L'utilisateur SalesRep1 ne voit que ses propres ventes

```
EXECUTE AS USER = 'SalesRep1';
SELECT * FROM Sales.Orders;
REVERT;
```

100 %

Results | Messages

	OrderID	SalesRep	Product	Quantity
1	1	SalesRep1	Valve	5
2	2	SalesRep1	Wheel	2
3	3	SalesRep1	Valve	4

Figure 4.h : Ventes de l'utilisateur SalesRep1

- L’utilisateur SalesRep2 voit uniquement ses ventes lui aussi

```sql
EXECUTE AS USER = 'SalesRep2';
SELECT * FROM Sales.Orders;
REVERT;
```

100 %

Results | Messages

	OrderID	SalesRep	Product	Quantity
1	4	SalesRep2	Bracket	2
2	5	SalesRep2	Wheel	5
3	6	SalesRep2	Seat	5

Figure 4.i : Ventes de l’utilisateur SalesRep2

- Par contre le Gestionnaire lui est capable de voir toutes les ventes

```sql
EXECUTE AS USER = 'Manager';
SELECT * FROM Sales.Orders;
REVERT;
```

100 %

Results | Messages

	OrderID	SalesRep	Product	Quantity
1	1	SalesRep1	Valve	5
2	2	SalesRep1	Wheel	2
3	3	SalesRep1	Valve	4
4	4	SalesRep2	Bracket	2
5	5	SalesRep2	Wheel	5
6	6	SalesRep2	Seat	5

Figure 4.j : Lignes vues par le Manager

NB: Une fois la stratégie de sécurité désactivée, les utilisateurs peuvent maintenant visualiser l'ensemble des ventes.

```sql
ALTER SECURITY POLICY SalesFilter
WITH (STATE = OFF);

EXECUTE AS USER = 'SalesRep1';
SELECT * FROM Sales.Orders;
REVERT;
```

100 %

Results | Messages

	OrderID	SalesRep	Product	Quantity
1	1	SalesRep1	Valve	5
2	2	SalesRep1	Wheel	2
3	3	SalesRep1	Valve	4
4	4	SalesRep2	Bracket	2
5	5	SalesRep2	Wheel	5
6	6	SalesRep2	Seat	5

Figure 4.k : Désactivation de la stratégie de sécurité

5) Le masquage dynamique de données (DDM)

Le masquage dynamique des données (DDM) limite l'exposition des données sensibles en les masquant aux utilisateurs sans privilège. Il peut être utilisé pour simplifier considérablement la conception et le codage de la sécurité dans votre application.

Le masquage dynamique des données permet d'empêcher les accès non autorisés à des données sensibles. Avec la fonctionnalité DDM, les données figurant dans la base de données ne sont pas modifiées. La fonctionnalité DDM est facile à utiliser avec des applications existantes, car les règles de masquage sont appliquées dans les résultats de la requête.

Il est possible de définir une règle de masquage sur une colonne d'une table, afin d'obfusquer les données qui y figurent. Quatre types de masques sont disponibles:

- **Default** : masquage complet en fonction des types de données des champs désignés.
- **Email** : méthode de masquage qui affiche la première lettre d'une adresse de messagerie et le suffixe de constante « .com », sous la forme d'une adresse de messagerie. Exemple : aXXX@XXXX.com
- **Aléatoire** : fonction de masquage aléatoire à utiliser sur tout type de données numérique pour masquer la valeur d'origine à l'aide d'une valeur aléatoire dans une plage spécifiée.
- **Chaîne personnalisée** : méthode de masquage qui affiche les première et dernière lettres, et ajoute une chaîne de remplissage personnalisée au milieu

- Pour cet exemple, on crée un schéma Data dans laquelle on crée une table Membership avec différents types de masques dynamiques de données.

```sql
CREATE SCHEMA Data;
GO


CREATE TABLE Data.Membership(
    MemberID        int IDENTITY(1,1) NOT NULL PRIMARY KEY CLUSTERED,
    FirstName        varchar(100) MASKED WITH (FUNCTION = 'partial(1, "xxxxx", 1)') NULL,
    LastName        varchar(100) NOT NULL,
    Phone            varchar(12) MASKED WITH (FUNCTION = 'default()') NULL,
    Email            varchar(100) MASKED WITH (FUNCTION = 'email()') NOT NULL,
    DiscountCode    smallint MASKED WITH (FUNCTION = 'random(1, 100)') NULL
    );


INSERT INTO Data.Membership (FirstName, LastName, Phone, Email, DiscountCode)
VALUES
('Roberto', 'Tamburello', '555.123.4567', 'RTamburello@contoso.com', 10),
('Janice', 'Galvin', '555.123.4568', 'JGalvin@contoso.com.co', 5),
('Shakti', 'Menon', '555.123.4570', 'SMenon@contoso.net', 50),
('Zheng', 'Mu', '555.123.4569', 'ZMu@contoso.net', 40);
```

100 %

Messages

```
(4 rows affected)

Completion time: 2022-12-22T18:50:08.9568995+00:00
```

Figure 5.a : Création du schéma **Data** et de la table **Membership**

- On crée un nouvel utilisateur et il reçoit l'autorisation **SELECT** sur le schéma où réside la table. Les requêtes exécutées en tant que MaskingTestUser affichent les données masquées.

```sql
CREATE USER MaskingTestUser WITHOUT LOGIN;

GRANT SELECT ON SCHEMA::Data TO MaskingTestUser;
GO
  -- impersonate for testing:
EXECUTE AS USER = 'MaskingTestUser';

SELECT * FROM Data.Membership;

REVERT;
```

100 %

Results | Messages

	MemberID	FirstName	LastName	Phone	Email	DiscountCode
1	1	Rxxxxxo	Tamburello	xxxx	RXXX@XXXX.com	42
2	2	Jxxxxxe	Galvin	xxxx	JXXX@XXXX.com	42
3	3	Sxxxxxi	Menon	xxxx	SXXX@XXXX.com	88
4	4	Zxxxxxg	Mu	xxxx	ZXXX@XXXX.com	34

Figure 5.b : Création de l'utilisateur

- Le résultat montre les masques en modifiant les données de:

FirstName	LastName	Phone	Email	DiscountCode
Roberto	Tamburello	555.123.4567	RTamburello@contoso.com	10
en				
Rxxxxxo	Tamburello	xxxx	Rxxxx@xxxxx.com	42

où le nombre dans DiscountCode est aléatoire pour chaque résultat de requête.

- L’octroi de l’autorisation **UNMASK** permet à MaskingTestUser d’afficher les données non masquées.

```sql
GRANT UNMASK TO MaskingTestUser;

EXECUTE AS USER = 'MaskingTestUser';

SELECT * FROM Data.Membership;

REVERT;
```

100 %

Results | Messages

	MemberID	FirstName	LastName	Phone	Email	DiscountCode
1	1	Roberto	Tamburello	555.123.4567	RTamburello@contoso.com	10
2	2	Janice	Galvin	555.123.4568	JGalvin@contoso.com.co	5
3	3	Shakti	Menon	555.123.4570	SMenon@contoso.net	50
4	4	Zheng	Mu	555.123.4569	ZMu@contoso.net	40

Figure 5.c : Révocation de privilège UNMASK

IX. Conclusion et Perspectives

La protection des données sous SQL Server regroupe plusieurs fonctionnalités et touche différents niveaux de la base de données à savoir la base de données elle-même ou une partie des tables qui la composent.

En sachant que chaque fonctionnalité de sécurité a un impact sur la performance de la base de données, il convient alors d’associer plusieurs d’entre elles pour protéger les données de manière optimale.

X. Bibliographie et Webographie

❑ Webographie

- W1. dbamantra [en ligne]. Publié le 20/01/2018 https://www.dbamantra.com/sql-server-encryption-to-secure-data/
- https://learn.microsoft.com/fr-fr/sql/relational-databases/security/
- http://blog.datafly.pro/post/2022/03/10/SQL-Server-et-protection-des-donnees/
- https://www.net4all.ch/publications/les-differents-modes-de-chiffrement-sql-server/